AI-Based Photo Restoration

Submitted by:

Syed Mazz Amir / 23108120
Ayesha Munir / 23108106
Faiza Khan / 23108109

Submitted to:

Mr. Muhammad Azhar

Department of Robotics & AI

SZABIST University Islamabad

Spring 2026

Table of Contents

1. Introduction 3
2. Features 3
3. Functional Requirements 4
4. Feasibility 4
5. Implementation 5
6. Conclusion 6
7. Contributions 6

1. Introduction

The Old or archived portrait photographs are often available only as grayscale scans that suffer from noise, scratches, and loss of color information. This project implements a CPU-friendly pipeline that maps a single-channel face image to a three-channel color reconstruction suitable for display and qualitative comparison with ground-truth color portraits. The core learning component is a convolutional encoder–decoder with skip connections. Skip links fuse multi-scale encoder features into the decoder so that spatial detail (edges, facial structure) is preserved instead of being washed out by a narrow bottleneck. Training uses paired grayscale and color images from the same subjects. A separate OpenCV stage refines the network output (contrast, luminance blending, and light sharpening) so results are more presentable for demos and reports. The system is implemented in Python using TensorFlow/Keras for the model and Streamlit for an interactive frontend; evaluation can be run from the command line with fixed metrics (PSNR, SSIM) and saved comparison strips.

2. Features

- **Skip-connection autoencoder:** Gray (1×128×128) → RGB (3×128×128) with encoder–decoder skip pathways at multiple resolutions for sharper structure than a plain bottleneck autoencoder.
- **Dual-stage restoration:** (1) neural colorization / reconstruction; (2) OpenCV post-processing for local detail and visual polish.
- **Dataset-driven training:** Learning from paired gray/ and color/ folders (same filenames).
- **Optional damage-aware training:** Synthetic degradation during training to better match scratched or noisy inputs.
- **Interactive UI:** Streamlit app for single-image upload or dataset samples, side-by-side previews, and batch restore with metrics and downloadable outputs.
- **Reproducible evaluation:** Script-based restore with mean PSNR/SSIM and saved comparison strips (input | network | enhanced | ground truth).

- **Checkpointing:** Best validation weights saved; optional timestamped backups to avoid losing good runs.

3. Functional Requirements

ID	Requirement	Description
FR1	Input	Accept grayscale images (e.g. JPG/PNG); resize to model input size (128×128).
FR2	Inference	Load trained skip-connection model; produce RGB in [0,1].
FR3	Post-process	Apply OpenCV pipeline (luminance handling, mild denoise, sharpening) to the network output.
FR4	Training	Build train/validation pipelines from paired folders; optimize combined L1 + SSIM loss; save best weights on validation.
FR5	Subset training	Support limiting the number of pairs (e.g. for faster runs) without requiring the full dataset every time.
FR6	UI	Provide single- and batch-restore flows with visible before/after and optional ZIP export.
FR7	Metrics	Compute PSNR and SSIM against color ground truth when available.
FR8	Artifacts	Persist model as .keras under artifacts/ and document paths for restore/UI.

4. Feasibility

- **Technical:**

  Encoder–decoder CNNs with skips are a standard, well-understood pattern for dense prediction (similar in spirit to hourglass / FPN-style fusion). TensorFlow and OpenCV are mature on Windows and run on CPU, avoiding GPU dependency for the project.

- **Data:**

  Feasibility assumes aligned pairs (same identity, same pose crop). If grays are heavily damaged while colors are clean, the task is harder; synthetic damage augmentation during training improves robustness without new hardware.

- **Scope:**

  128×128 keeps memory and training time manageable. Quality trades off against resolution; super-resolution is out of scope.

- **Risks:**

  Very strong scratches or domain shift (non-face images) will limit quality; the design mitigates this with augmentation and post-processing, not with a second backbone architecture.

5. Implementation

- **Model:**

  Repeated encoder blocks (3×3 conv, ReLU, stride-2 conv) produce tensors at 64×64, 32×32, 16×16, and 8×8; a small bottleneck stack operates at 8×8. The decoder applies UpSampling2D(×2) and concatenates the matching encoder feature maps, then two 3×3 conv layers per stage—reducing blur compared to transpose-convolution-only decoders. Final 1×1 conv + sigmoid yields RGB.

- **Loss:**

  0.75 × L1 + 0.25 × (1 − SSIM) to balance pixel accuracy and perceptual structure.

- **Data:**

  build_datasets reads paired JPGs, normalizes to [0,1], and optionally applies damage augmentation.

- **Training (train.py):**

  baseline trains only this skip model; ModelCheckpoint monitors val_loss; learning rate reduction and early stopping improve stability.

- **Inference (restore.py / app.py):**

  Load compile=False, run forward pass, then postprocess_with_opencv and optional scratch/color helpers for the demo pipeline.

- **Frontend:**

  Streamlit loads the baseline checkpoint from artifacts/restorer_autoencoder.keras (with file mtime in the cache key so replaced weights reload correctly).

6. Conclusion

This project restores grayscale face images to color using one model: a convolutional encoder–decoder with skip connections at 128×128. Skip links keep spatial detail that a plain bottleneck would lose. Training uses paired gray/color data and an L1 + SSIM loss; OpenCV post-processing improves how results look in demos. The setup runs on CPU, supports subset training and damage augmentation for noisy scans, and uses Streamlit plus scripts for evaluation (PSNR/SSIM) and checkpointing.

Limits: fixed low resolution, sensitivity to bad or misaligned pairs, and focus on faces like the training set. Still, the work shows that a skip-linked autoencoder plus light post-processing is enough for clear, coursework-scale grayscale-to-color restoration.

7. Contributions

Name	**Registration**	**Contribution**
Syed Mazz Amir	23108120	Code
Ayesha Munir	23108106	Code/PPT
Faiza Khan	23108109	Code/Documentation